

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
Национальный исследовательский университет ИТМО**

**Отчет по лабораторной работе №6
«ВВЕДЕНИЕ в СУБД MongoDB. Работа с БД в СУБД MongoDB »
по дисциплине «Проектирование и реализация баз данных»**

Обучающийся Буй Тхук Хуен

Факультет прикладной информатики - **Группа** K3239

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2023

Преподаватель Говорова Марина Михайловна

Белов Александр Олегович

Санкт-Петербург
2026 г.

1. Цель работы

Изучить основы работы с СУБД MongoDB, научиться создавать и удалять базы данных и коллекции, а также выполнять основные команды MongoDB Shell. Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

2. Практическое задание

- Установка MongoDB. Начало работы с БД
- Работа с БД в СУБД MongoDB

3. Выполнение

3.1 Работа с БД

Проверка работоспособности системы запуском клиента mongo.

```
> show dbs
< admin   40.00 KiB
  config  12.00 KiB
  local   40.00 KiB
> db.stats()
< {
  db: 'admin',
  collections: Long('1'),
  views: Long('0'),
  objects: Long('1'),
  avgObjSize: 59,
  dataSize: 59,
  storageSize: 20480,
  indexes: Long('1'),
  indexSize: 20480,
  totalSize: 40960,
  scaleFactor: Long('1'),
  fsUsedSize: 35234635776,
  fsTotalSize: 187597582336,
  ok: 1
}
```

Для перехода к новой базе данных была выполнена команда: use learn.

→ MongoDB автоматически создаёт базу данных после добавления данных. Создание коллекции unicorns и добавление документа

В коллекцию unicorns был добавлен документ:

```
db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})
```

→ После вставки документа база данных и коллекция были созданы автоматически.

```
>_MONGOSH
> use learn
< switched to db learn
> db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})
< {
  acknowledged: true,
  insertedId: ObjectId('6a185aebcaf8cae267f29e5e')
}
> show dbs
< admin   40.00 KiB
  config  48.00 KiB
  learn   8.00 KiB
  local   72.00 KiB
> show collections
< unicorns
> db.createCollection("articles")
< { ok: 1 }
> db.createCollection("logs", {capped: true, size: 64000} )
< { ok: 1 }
> db.unicorns.renameCollection("pages")
< { ok: 1 }
```

- После добавления документа была снова выполнена команда: `show dbs`. В списке появилась база данных `learn`.
- Для просмотра коллекций использовалась команда: `show collections`
- Коллекция была создана вручную при помощи команды:
`db.createCollection("articles")`
- Была создана ограниченная коллекция с фиксированным размером:
`db.createCollection( "logs", {capped: true, size: 64000})`
Такая коллекция автоматически удаляет старые записи при превышении установленного размера.
- Коллекция `unicorns` была переименована в `pages`:
`db.unicorns.renameCollection("pages")`
- Для получения информации о коллекции была выполнена команда: `db.pages.stats()`. Была получена информация о размере коллекции, количестве документов и индексах.

>_MONGOSH



> db.pages.stats()

< {

ok: 1,

capped: false,

wiredTiger: {

metadata: { formatVersion: 1 },

creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata=(formatVersi
type: 'file',

uri: 'statistics:table:collection-d5061f04-25ff-4f19-b721-8007d09d44de',

version: 'WiredTiger 12.0.0: (November 15, 2024)',

autocommit: {

'retries for readonly operations': 0,

'retries for update operations': 0

},

backup: {

'total modified incremental blocks with compressed data': 0,

'total modified incremental blocks without compressed data': 0

},

'block-disagg': {

'Bytes read from the shared history store in SLS': 0,

'Bytes written to the shared history store in SLS': 0,

'Disaggregated block manager get': 0,

'Disaggregated block manager get cold page': 0,

'Disaggregated block manager get from the shared history store in SLS': 0,

'Disaggregated block manager page discard calls': 0,

'Disaggregated block manager put': 0,

'Disaggregated block manager put cold page': 0,

'Disaggregated block manager put to the shared history store in SLS': 0,

'Disaggregated block manager read ahead of materialization frontier': 0

},

'block-manager': {

'allocations requiring file extension': 4,

'blocks allocated': 4,

'blocks freed': 0,

'checkpoint size': 4096,

'file allocation unit size': 4096,

'file bytes available for reuse': 0,

>_MONGOSH

1

```
'number of times overflow removed value is read': 0,  
'race to read prepared update retry': 0,  
'rollback to stable applied btrees': 0,  
'rollback to stable history store keys that would have been swept in non-dryrun mode'  
'rollback to stable history store records with stop timestamps older than newer recor  
'rollback to stable inconsistent checkpoint': 0,  
'rollback to stable keys removed': 0,  
'rollback to stable keys restored': 0,  
'rollback to stable keys that would have been removed in non-dryrun mode': 0,  
'rollback to stable keys that would have been restored in non-dryrun mode': 0,  
'rollback to stable restored tombstones from history store': 0,  
'rollback to stable restored updates from history store': 0,  
'rollback to stable skipped btrees': 0,  
'rollback to stable skipping delete rle': 0,  
'rollback to stable skipping stable rle': 0,  
'rollback to stable sweeping history store keys': 0,  
'rollback to stable tombstones from history store that would have been restored in no  
'rollback to stable updates from history store that would have been restored in non-d  
'rollback to stable updates removed from history store': 0,  
'rollback to stable updates that would have been removed from history store in non-dr  
'update conflicts': 0  
}  
},  
sharded: false,  
size: 65,  
count: 1,  
numOrphanDocs: 0,  
storageSize: 20480,  
totalIndexSize: 20480,  
totalSize: 40960,  
indexSizes: { _id_: 20480 },  
avgObjSize: 65,  
ns: 'learn.pages',  
nindexes: 1,  
scaleFactor: 1  
}
```

– Коллекция pages была удалена командой: db.pages.drop()

```
> db.pages.drop()  
< true  
> db.system.namespaces.find()  
<  
> db.system.indexes.find()  
<  
learn> |
```

– Для удаления базы данных была выполнена команда: `db.dropDatabase()`
После этого база данных `learn` была удалена.

```
> db.dropDatabase()
< { ok: 1, dropped: 'learn' }
```

3.2. ВСТАВКА ДОКУМЕНТОВ В КОЛЛЕКЦИЮ

В коллекцию были добавлены документы с помощью методов `insertOne()`.

```
> use learn
< already on db learn
> db.unicorns.insertOne({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63})
< {
  acknowledged: true,
  insertedId: ObjectId('6a1865d4caf8cae267f29e5f')
}
> db.unicorns.insertOne({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43})
< {
  acknowledged: true,
  insertedId: ObjectId('6a18660ecaf8cae267f29e60')
}
> db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
< DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a186651caf8cae267f29e69')
  }
}
}
```

Также был использован второй способ вставки документа через промежуточную переменную `document`.

```
> document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
< {
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
> db.unicorns.insertOne(document)
< {
  acknowledged: true,
  insertedId: ObjectId('6a1866c7caf8cae267f29e6a')
}
```

После выполнения команды `find()` было проверено содержимое коллекции.

```

> db.unicorns.find()
< {
  _id: ObjectId('6a1865d4caf8cae267f29e5f'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
  _id: ObjectId('6a18660ecaf8cae267f29e60'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a186651caf8cae267f29e61'),
  name: 'Unicrom',
  loves: [
    'energon',
    'redbull'
  ],
  weight: 984,
  gender: 'm',
  vampires: 182
}

```

```

{
  _id: ObjectId('6a186651caf8cae267f29e62'),
  name: 'Roooooodles',
  loves: [
    'apple'
  ],
  weight: 575,
  gender: 'm',
  vampires: 99
}
{
  _id: ObjectId('6a186651caf8cae267f29e63'),
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80
}
{
  _id: ObjectId('6a186651caf8cae267f29e64'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 733,
  gender: 'f',
  vampires: 40
}

```

```

> db.unicorns.countDocuments()
< 12

```

Была создана база данных learn и коллекция unicorns.

3.3 ВЫБОРКА ДАННЫХ ИЗ БД

3.3.1 SELECT / FIND / SORT / LIMIT

– Вывод списка самцов единорогов с сортировкой по имени

```
> db.unicorns.find({ gender: 'm'})
< {
  _id: ObjectId('6a1865d4caf8cae267f29e5f'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
  _id: ObjectId('6a186651caf8cae267f29e61'),
  name: 'Unicrom',
  loves: [
    'energon',
    'redbull'
  ],
  weight: 984,
  gender: 'm',
  vampires: 182
}
{
  _id: ObjectId('6a186651caf8cae267f29e62'),
  name: 'Rooooooodles',
  loves: [
    'apple'
  ],
  weight: 575,
  gender: 'm',
  vampires: 99
}
{
  _id: ObjectId('6a186651caf8cae267f29e65'),
  name: 'Kenny',
  loves: [
```


– Вывод списка самок единорогов с сортировкой по имени

```
> db.unicorns.find({gender:'m'}).sort({name:1})
< {
  _id: ObjectId('6a1866c7caf8cae267f29e6a'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a1865d4caf8cae267f29e5f'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
  _id: ObjectId('6a186651caf8cae267f29e65'),
  name: 'Kenny',
  loves: [
    'grape',
    'lemon'
  ],
  weight: 690,
  gender: 'm',
  vampires: 39
}
{
  _id: ObjectId('6a186651caf8cae267f29e68'),
  name: 'Pilot',
  loves: [
```

>_MONGOSH

> db.unicorns.find({gender:'f'}).sort({name:1})

< {

 _id: ObjectId('6a18660ecaf8cae267f29e60'),

 name: 'Aurora',

 loves: [

 'carrot',

 'grape'

],

 weight: 450,

 gender: 'f',

 vampires: 43

}

{

 _id: ObjectId('6a186651caf8cae267f29e64'),

 name: 'Ayna',

 loves: [

 'strawberry',

 'lemon'

],

 weight: 733,

 gender: 'f',

 vampires: 40

}

{

 _id: ObjectId('6a186651caf8cae267f29e67'),

 name: 'Leia',

 loves: [

 'apple',

 'watermelon'

],

 weight: 601,

 gender: 'f',

 vampires: 33

}

{

 _id: ObjectId('6a186651caf8cae267f29e69'),

 name: 'Nimue',

– Ограничение списка самок первыми тремя особями

```
> db.unicorns.find({gender:'f'}).sort({name:1}).limit(3)
< {
  _id: ObjectId('6a18660ecaf8cae267f29e60'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a186651caf8cae267f29e64'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 733,
  gender: 'f',
  vampires: 40
}
{
  _id: ObjectId('6a186651caf8cae267f29e67'),
  name: 'Leia',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 601,
  gender: 'f',
  vampires: 33
}
```

– Поиск самок единорогов, любящих carrot

```
> db.unicorns.find({gender:'f', loves:'carrot'})
< {
  _id: ObjectId('6a18660ecaf8cae267f29e60'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a186651caf8cae267f29e63'),
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80
}
{
  _id: ObjectId('6a186651caf8cae267f29e69'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
```

– Поиск одной самки единорога с помощью метода findOne()

```
> db.unicorns.findOne({gender:'f', loves:'carrot'})
< {
  _id: ObjectId('6a18660ecaf8cae267f29e60'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

– Ограничение выборки одной записью с помощью метода limit()

```
> db.unicorns.find({gender:'f', loves:'carrot'}).limit(1)
< {
  _id: ObjectId('6a18660ecaf8cae267f29e60'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

– Вывод списка самцов единорогов без полей loves и gender

```
> db.unicorns.find({gender:'m'}, {_id:0, loves:0, gender:0})
< {
  name: 'Horny',
  weight: 600,
  vampires: 63
}
{
  name: 'Unicrom',
  weight: 984,
  vampires: 182
}
{
  name: 'Rooooooodles',
  weight: 575,
  vampires: 99
}
{
  name: 'Kenny',
  weight: 690,
  vampires: 39
}
{
  name: 'Raleigh',
  weight: 421,
  vampires: 2
}
{
  name: 'Pilot',
  weight: 650,
  vampires: 54
}
{
  name: 'Dunx',
  weight: 704,
  vampires: 165
}
```

– Вывод списка единорогов в обратном порядке добавления

```
> db.unicorns.find({gender:'m'}, {_id:0, loves:0, gender:0})
< {
  name: 'Horny',
  weight: 600,
  vampires: 63
}
{
  name: 'Unicrom',
  weight: 984,
  vampires: 182
}
{
  name: 'Roooooodles',
  weight: 575,
  vampires: 99
}
{
  name: 'Kenny',
  weight: 690,
  vampires: 39
}
{
  name: 'Raleigh',
  weight: 421,
  vampires: 2
}
{
  name: 'Pilot',
  weight: 650,
  vampires: 54
}
{
  name: 'Dunx',
  weight: 704,
  vampires: 165
}
```

– Вывод первого предпочтения единорогов с использованием оператора \$slice

```
>_MONGOSH
> db.unicorns.find({}, {_id:0, name:1, loves:{$slice:1}})
< {
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{
  name: 'Aurora',
  loves: [
    'carrot'
  ]
}
{
  name: 'Unicrom',
  loves: [
    'energon'
  ]
}
{
  name: 'Rooooooodles',
  loves: [
    'apple'
  ]
}
{
  name: 'Solnara',
  loves: [
    'apple'
  ]
}
{
  name: 'Ayna',
  loves: [
    'strawberry'
  ]
}
```


3.3.2 Проекция полей

– Вывод списка самцов единорогов без полей loves и gender

```
> db.unicorns.find({gender:'m'}, {_id:0, loves:0, gender:0})
< {
  name: 'Horny',
  weight: 600,
  vampires: 63
}
{
  name: 'Unicrom',
  weight: 984,
  vampires: 182
}
{
  name: 'Roooooodles',
  weight: 575,
  vampires: 99
}
{
  name: 'Kenny',
  weight: 690,
  vampires: 39
}
{
  name: 'Raleigh',
  weight: 421,
  vampires: 2
}
{
  name: 'Pilot',
  weight: 650,
  vampires: 54
}
{
  name: 'Dunx',
  weight: 704,
  vampires: 165
}
```

Выборка данных была выполнена с использованием проекции полей. Из результата были исключены поля loves, gender и _id с помощью значения 0.

3.3.3 Сортировка по порядку добавления

– Вывод списка единорогов в обратном порядке добавления

Для сортировки документов в обратном порядке добавления использовался параметр `$natural:-1`.

Также была ограничена выборка первыми пятью документами с помощью метода `limit()`.

```
> db.unicorns.find().sort({$natural:-1})
< {
  _id: ObjectId('6a1866c7caf8cae267f29e6a'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a186651caf8cae267f29e69'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
{
  _id: ObjectId('6a186651caf8cae267f29e68'),
  name: 'Pilot',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 650,
  gender: 'm',
  vampires: 54
}
```

```
{
  _id: ObjectId('6a186651caf8cae267f29e67'),
  name: 'Leia',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 601,
  gender: 'f',
  vampires: 33
}
{
  _id: ObjectId('6a186651caf8cae267f29e66'),
  name: 'Raleigh',
  loves: [
    'apple',
    'sugar'
  ],
  weight: 421,
  gender: 'm',
  vampires: 2
}
{
  _id: ObjectId('6a186651caf8cae267f29e65'),
  name: 'Kenny',
  loves: [
    'grape',
    'lemon'
  ],
  weight: 690,
  gender: 'm',
  vampires: 39
}
```

3.3.4 Использование оператора \$slice

– Вывод первого предпочтения единорогов без идентификатора

Для работы с массивами был использован оператор `$slice`.

С его помощью был получен первый и последний элементы массива `loves`.

Также из результата был исключен идентификатор `_id`.

```
> db.unicorns.find({}, {_id:0, name:1, loves:{$slice:1}})
< {
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{
  name: 'Aurora',
  loves: [
    'carrot'
  ]
}
{
  name: 'Unicrom',
  loves: [
    'energon'
  ]
}
{
  name: 'Rooooooodles',
  loves: [
    'apple'
  ]
}
{
  name: 'Solnara',
  loves: [
    'apple'
  ]
}
{
  name: 'Ayna',
  loves: [
    'strawberry'
  ]
}
```

– Оператор \$slice:-1 используется для получения последнего элемента массива.

```
> db.unicorns.find({}, {_id:0, name:1, loves:{$slice: [-1,1]}})
< {
  name: 'Horny',
  loves: [
    'papaya'
  ]
}
{
  name: 'Aurora',
  loves: [
    'grape'
  ]
}
{
  name: 'Unicrom',
  loves: [
    'redbull'
  ]
}
{
  name: 'Rooooooodles',
  loves: [
    'apple'
  ]
}
{
  name: 'Solnara',
  loves: [
    'chocolate'
  ]
}
{
  name: 'Ayna',
  loves: [
    'lemon'
  ]
}
```

– Оператор `$slice: [-1,1]` также возвращает последний элемент массива, однако в данном случае дополнительно указывается начальная позиция и количество извлекаемых элементов.

Таким образом, первый вариант является более простым способом получения последнего элемента массива, а второй вариант позволяет более гибко управлять выборкой элементов массива.

3.4 Логические операторы

3.4.1 Вывод списка самок единорогов весом от 500 до 700 кг без идентификатора

```

> db.unicorns.find({gender:'f', weight:{$gte:500, $lte:700}}, {_id:0})
< {
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80
}
{
  name: 'Leia',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 601,
  gender: 'f',
  vampires: 33
}
{
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}

```

- \$gte → больше или равно
- \$lte → меньше или равно

Запрос выводит самок единорогов с весом от 500 до 700 кг включительно.

Поле `_id` было исключено из результата выборки.

3.4.2 Использование операторов \$all и \$gte

– Вывод списка самцов единорогов весом более 500 кг, любящих grape и lemon

```
> db.unicorns.find({gender:'m', weight:{$gte:500}, loves:{$all:['grape','lemon']}}, {_id:0})
< {
  name: 'Kenny',
  loves: [
    'grape',
    'lemon'
  ],
  weight: 690,
  gender: 'm',
  vampires: 39
}
```

- \$gte:500 → вес больше или равен 500 кг
- \$all → массив должен содержать одновременно все указанные элементы

Запрос выводит самцов единорогов весом более 500 кг, которые одновременно любят grape и lemon.

Поле `_id` исключено из результата выборки.

3.4.3 Использование оператора \$exists

– Поиск единорогов без ключа vampires

```
> db.unicorns.find({vampires:{$exists:false}}, {_id:0})
< {
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
```

Оператор `$exists:false` используется для поиска документов, в которых отсутствует поле `vampires`.

Запрос выводит всех единорогов, у которых не определён ключ `vampires`

3.4.4 Использование оператора \$slice

Вывод списка имён самцов единорогов и их первого предпочтения

```
> db.unicorns.find({gender:'m'}, {_id:0, name:1, loves:{$slice:1}}).sort({name:1})
< {
  name: 'Dunx',
  loves: [
    'grape'
  ]
}
{
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{
  name: 'Kenny',
  loves: [
    'grape'
  ]
}
{
  name: 'Pilot',
  loves: [
    'apple'
  ]
}
{
  name: 'Raleigh',
  loves: [
    'apple'
  ]
}
{
  name: 'Rooooooodles',
  loves: [
    'apple'
  ]
}
```

Оператор `$slice:1` используется для получения первого элемента массива `loves`.

Список самцов единорогов был отсортирован по имени в лексикографическом порядке.

Из результата выборки был исключён идентификатор `_id`.

3.5 ЗАПРОС К ВЛОЖЕННЫМ ОБЪЕКТАМ

3.5.1 Запросы к вложенным объектам

Создание коллекции `towns`

```
> db.towns.insertMany([{name:"Punxsutawney", population:6200, last_sensus:ISODate("2008-01-31"),
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a187321caf8cae267f29e6b'),
    '1': ObjectId('6a187321caf8cae267f29e6c'),
    '2': ObjectId('6a187321caf8cae267f29e6d')
  }
}
```

— Запрос городов с независимыми мэрами

```
> db.towns.find({"mayor.party":"I"}, {_id:0, name:1, mayor:1})
< {
  name: 'New York',
  mayor: {
    name: 'Michael Bloomberg',
    party: 'I'
  }
}
```

— Запрос городов с беспартийными мэрами

```
> db.towns.find({"mayor.party":{"$exists:false"}}, {_id:0, name:1, mayor:1})
< {
  name: 'Punxsutawney',
  mayor: {
    name: 'Jim Wehrle'
  }
}
```

Для обращения к вложенным объектам использовался оператор точка.

Поле mayor.party позволяет обращаться к свойству party внутри вложенного объекта mayor.

Оператор \$exists:false использовался для поиска документов, в которых поле party отсутствует.

3.5.2 Использование JavaScript и курсоров

Создание функции для вывода самцов единорогов

```
> fn = function(obj){ return obj.gender=='m'; }
< [Function: fn]
```

Создание курсора

```
> var cursor = db.unicorns.find({gender:'m'}).sort({name:1}).limit(2)
```

Вывод результата с помощью forEach


```
> cursor.forEach(function(obj){print(obj.name)})  
< Dunx  
< Horny
```

Для формирования запроса использовалась функция JavaScript.

Был создан курсор с использованием методов find(), sort() и limit().

Для вывода результата использовался метод forEach(), который позволяет перебрать все документы курсора.

3.6. Агрегированные запросы

3.6.1 Подсчёт количества самок единорогов весом от 500 до 600 кг

Для подсчёта количества документов использовался метод count().

Запрос выводит количество самок единорогов весом от 500 до 600 кг включительно.

```
> db.unicorns.find({gender:'f', weight:{$gte:500, $lte:600}}).count()  
< 2
```

3.6.2 Вывод списка предпочтений

Вывод уникальных предпочтений единорогов

```
> db.unicorns.distinct("loves")  
< [  
  'apple',      'carrot',  
  'chocolate', 'energon',  
  'grape',      'lemon',  
  'papaya',     'redbull',  
  'strawberry', 'sugar',  
  'watermelon'  
]
```

Метод distinct() используется для получения уникальных значений массива loves.

3.6.3 Подсчёт количества самцов и самок

Подсчёт количества единорогов обоих полов

```
> db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}}])  
< {  
  _id: 'f',  
  count: 5  
}  
{  
  _id: 'm',  
  count: 7  
}
```

Метод aggregate() использовался для группировки документов по полю gender.

Оператор \$sum подсчитывает количество документов в каждой группе.

3.7 РЕДАКТИРОВАНИЕ ДАННЫХ

3.7.1 Использование метода save

Добавление нового единорога Barny

Метод save() использовался для добавления нового документа в коллекцию unicorns.

```
> db.unicorns.insertOne({name:'Barny', loves:['grape'], weight:340, gender:'m'})
< {
  acknowledged: true,
  insertedId: ObjectId('6a187634caf8cae267f29e6e')
}
```

После добавления нового единорога содержимое коллекции было проверено с помощью метода find().

```
> db.unicorns.find({name:'Barny'})
< {
  _id: ObjectId('6a187634caf8cae267f29e6e'),
  name: 'Barny',
  loves: [
    'grape'
  ],
  weight: 340,
  gender: 'm'
}
```

3.7.2 Обновление документа с помощью update

Изменение информации о единороге Ayna

```
> db.unicorns.updateOne({name:'Ayna'}, {$set:{weight:800, vampires:51}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Оператор \$set использовался для изменения значений полей weight и vampires.

```
> db.unicorns.find({name:'Ayna'})
< {
  _id: ObjectId('6a186651caf8cae267f29e64'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 800,
  gender: 'f',
  vampires: 51
}
```

3.7.3 Добавление нового предпочтения

Изменение информации о Raleigh

```
> db.unicorns.updateOne({name:'Raleigh'}, {$push:{loves:'redbull'}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Оператор \$push использовался для добавления нового элемента в массив loves.

```
> db.unicorns.find({name:'Raleigh'})
< {
  _id: ObjectId('6a186651caf8cae267f29e66'),
  name: 'Raleigh',
  loves: [
    'apple',
    'sugar',
    'redbull'
  ],
  weight: 421,
  gender: 'm',
  vampires: 2
}
```

3.7.4 Увеличение количества vampires

Увеличение количества убитых вампиров на 5

```
> db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
```

```
> db.unicorns.find({gender: 'm'})
< {
  _id: ObjectId('6a1865d4caf8cae267f29e5f'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 68
}
{
  _id: ObjectId('6a186651caf8cae267f29e61'),
  name: 'Unicrom',
  loves: [
    'energon',
    'redbull'
  ],
  weight: 984,
  gender: 'm',
  vampires: 187
}
{
  _id: ObjectId('6a186651caf8cae267f29e62'),
  name: 'Rooooooodles',
  loves: [
    'apple'
  ],
  weight: 575,
  gender: 'm',
  vampires: 104
}
```

Оператор \$inc использовался для увеличения числового значения поля vampires на 5 у всех самцов единорогов.

3.7.5 Удаление поля из документа

Удаление поля vampires у единорога Pilot

```
> db.unicorns.updateOne({name:'Pilot'}, {$unset:{vampires:''}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.unicorns.find({name:'Pilot'})
< {
  _id: ObjectId('6a186651caf8cae267f29e68'),
  name: 'Pilot',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 650,
  gender: 'm'
}
```

Оператор \$unset использовался для удаления поля vampires из документа.

3.7.6 Удаление документа

Удаление единорога Barny

```
> db.unicorns.deleteOne({name:'Barny'})
< {
  acknowledged: true,
  deletedCount: 1
}
> db.unicorns.find({name:'Barny'})
<
learn>
```

Метод deleteOne() использовался для удаления одного документа из коллекции unicorns.

3.7.7 Удаление нескольких документов

Удаление всех единорогов без поля vampires

```
> db.unicorns.deleteMany({vampires:{$exists:false}})
< {
  acknowledged: true,
  deletedCount: 2
}
```

Метод deleteMany() использовался для удаления всех документов, в которых отсутствует поле vampires.

```
> db.unicorns.find()
< {
  _id: ObjectId('6a1865d4caf8cae267f29e5f'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 68
}
{
  _id: ObjectId('6a18660ecaf8cae267f29e60'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a186651caf8cae267f29e61'),
  name: 'Unicrom',
  loves: [
    'energon',
    'redbull'
  ],
  weight: 984,
  gender: 'm',
  vampires: 187
}
```

```
{
  _id: ObjectId('6a186651caf8cae267f29e62'),
  name: 'Rooooooodles',
  loves: [
    'apple'
  ],
  weight: 575,
  gender: 'm',
  vampires: 104
}
{
  _id: ObjectId('6a186651caf8cae267f29e63'),
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80
}
{
  _id: ObjectId('6a186651caf8cae267f29e64'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 800,
  gender: 'f',
  vampires: 51
}
```

3.8 ССЫЛКИ И РАБОТА С ИНДЕКСАМИ В БАЗЕ ДАННЫХ MONGODB

3.8.1 Создание связанных документов (DBRef)

– Создание коллекции employees

```
> db.employees.insertMany([{name:'John', position:'manager'}, {name:'Anna', position:'developer'}]
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a187880caf8cae267f29e6f'),
    '1': ObjectId('6a187880caf8cae267f29e70')
  }
}
```

```
> db.employees.find()
< {
  _id: ObjectId('6a187880caf8cae267f29e6f'),
  name: 'John',
  position: 'manager'
}
{
  _id: ObjectId('6a187880caf8cae267f29e70'),
  name: 'Anna',
  position: 'developer'
}
```

– Создание коллекции departments со ссылкой на сотрудника

Для создания связи между документами использовался объект DBRef.

Коллекция departments содержит ссылку на документ из коллекции employees.

```
> db.departments.insertOne({name:'IT', chief:DBRef('employees', ObjectId('6a187880caf8cae267f29e6f'))})
< {
  acknowledged: true,
  insertedId: ObjectId('6a1878d5caf8cae267f29e71')
}
> db.departments.insertOne({name:'IT', chief:DBRef('employees', ObjectId('6a187880caf8cae267f29e70'))})
< {
  acknowledged: true,
  insertedId: ObjectId('6a1878decaf8cae267f29e72')
}
```

Просмотр коллекции departments

```
> db.departments.find()
< {
  _id: ObjectId('6a1878d5caf8cae267f29e71'),
  name: 'IT',
  chief: DBRef('employees', ObjectId('6a187880caf8cae267f29e6f'))
}
{
  _id: ObjectId('6a1878decaf8cae267f29e72'),
  name: 'IT',
  chief: DBRef('employees', ObjectId('6a187880caf8cae267f29e70'))
}
```

3.8.2 Создание индекса по полю name

Индекс был создан по полю name для ускорения поиска документов

```
> db.unicorns.createIndex({name:1})
< name_1
> db.unicorns.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1' }
]
```

Удаление индекса name

```
> db.unicorns.dropIndex("name_1")
< { nIndexesWas: 2, ok: 1 }
> db.unicorns.getIndexes()
< [ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

Попытка удаления индекса _id

```
> db.unicorns.dropIndex("_id")
✖ > MongoServerError[InvalidOptions]: cannot drop _id index
```

→ Попытка удаления индекса _id_ завершилась ошибкой, так как данный индекс является системным и не может быть удалён.

3.8.3 Использование explain()

Анализ выполнения запроса:

```
> db.unicorns.find({name:'Horny'}).explain('executionStats')
```

Метод explain() использовался для анализа выполнения запроса.

Параметр executionStats позволяет получить информацию о производительности запроса и использовании индексов.

```
{
  explainVersion: '1',
```



```
queryPlanner: {
  namespace: 'learn.unicorns',
  parsedQuery: {
    name: {
      '$eq': 'Horny'
    }
  },
  indexFilterSet: false,
  queryHash: 'F4DDDCDC',
  planCacheShapeHash: 'F4DDDCDC',
  planCacheKey: '34C29116',
  optimizationTimeMillis: 4,
  maxIndexedOrSolutionsReached: false,
  maxIndexedAndSolutionsReached: false,
  maxScansToExplodeReached: false,
  prunedSimilarIndexes: false,
  winningPlan: {
    isCached: false,
    stage: 'FETCH',
    nss: 'learn.unicorns',
    inputStage: {
      stage: 'IXSCAN',
      nss: 'learn.unicorns',
      keyPattern: {
        name: 1
      },
      indexName: 'name_1',
      isMultiKey: false,
      multiKeyPaths: {
        name: []
      },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        name: [
          ["Horny", "Horny"]
        ]
      }
    }
  },
},
```

```
rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 6,
  totalKeysExamined: 1,
  totalDocsExamined: 1,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 1,
    executionTimeMillisEstimate: 0,
    works: 2,
    advanced: 1,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    nss: 'learn.unicorns',
    docsExamined: 1,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 2,
      advanced: 1,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      nss: 'learn.unicorns',
      keyPattern: {
        name: 1
      },
      indexName: 'name_1',
      isMultiKey: false,
      multiKeyPaths: {
        name: []
      },
    },
  },
}
```

```
isUnique: false,
isSparse: false,
isPartial: false,
indexVersion: 2,
direction: 'forward',
indexBounds: {
  name: [
    ["Horny", "Horny"]
  ],
},
keysExamined: 1,
seeks: 1,
dupsTested: 0,
dupsDropped: 0,
peakTrackedMemBytes: 0
}
},
```

queryShapeHash:

'70472B62FCC4AE7AD2CDF8DFA78782C2FB38FFB49F5ADA6A628192975DE5D6E4'

```
,
command: {
  find: 'unicorns',
  filter: {
    name: 'Horny'
  },
  '$db': 'learn'
},
```

```
serverInfo: {
  host: 'bluemurmur',
  port: 27017,
  version: '8.3.2',
  gitVersion: '89f54eb32e217d2f7bfcad50e5919f3033bb9611'
},
```

```
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
```

```
internalQueryFrameworkControl: 'trySbeRestricted',
internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
```

3.9. План запроса

3.9.1 Создание коллекции numbers

Создание объемной коллекции

```
> for(i=0;i<1000000;i++){db.numbers.insertOne({value:i})}
< {
  acknowledged: true,
  insertedId: ObjectId('6a187f82caf8cae267f42512')
}
```

Выбор последних четырёх документов

```
> db.numbers.find().sort({value:-1}).limit(4)
< {
  _id: ObjectId('6a187f82caf8cae267f42512'),
  value: 99999
}
{
  _id: ObjectId('6a187f82caf8cae267f42511'),
  value: 99998
}
{
  _id: ObjectId('6a187f82caf8cae267f42510'),
  value: 99997
}
{
  _id: ObjectId('6a187f82caf8cae267f4250f'),
  value: 99996
}
```

Анализ плана запроса без индекса

```
> db.numbers.find().sort({value:-1}).limit(4).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: {
        value: -1
      },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: {
        stage: 'COLLSCAN',
        nss: 'learn.numbers',
        direction: 'forward'
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 89,
    totalKeysExamined: 0,
```

```
totalDocsExamined: 100000,
executionStages: {
  isCached: false,
  stage: 'SORT',
  nReturned: 4,
  executionTimeMillisEstimate: 84,
  works: 100006,
  advanced: 4,
  needTime: 100001,
  needYield: 0,
  saveState: 4,
  restoreState: 4,
  isEOF: 1,
  sortPattern: {
    value: -1
  },
  memLimit: 104857600,
  limitAmount: 4,
  type: 'simple',
  totalDataSizeSorted: 260,
  usedDisk: false,
  spills: 0,
  spilledRecords: 0,
  spilledBytes: 0,
  spilledDataStorageSize: 0,
  peakTrackedMemBytes: 260,
  inputStage: {
    stage: 'COLLSCAN',
    nReturned: 100000,
    executionTimeMillisEstimate: 41,
    works: 100001,
    advanced: 100000,
    needTime: 0,
    needYield: 0,
    saveState: 4,
    restoreState: 4,
    isEOF: 1,
```

```

      nss: 'learn.numbers',
      direction: 'forward',
      docsExamined: 100000
    }
  },
  queryShapeHash: 'A1C8CFCE9F916AB3A6ABCC8ABBB22506390F4D987582D3F926F0F2B36CA78396',
  command: {
    find: 'numbers',
    filter: {},
    sort: {
      value: -1
    },
    limit: 4,
    '$db': 'learn'
  },
  serverInfo: {
    host: 'bluemurmur',
    port: 27017,
    version: '8.3.2',
    gitVersion: '89f54eb32e217d2f7bfcad50e5919f3033bb9611'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}

```

executionTimeMillis

```
executionTimeMillis: 89,
```

Создание индекса для value

```

> db.numbers.createIndex({value:1})
< value_1

```

Просмотр индексов коллекции numbers

```
> db.numbers.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

Повторное выполнение запроса

```
> db.numbers.find().sort({value:-1}).limit(4)
< {
  _id: ObjectId('6a187f82caf8cae267f42512'),
  value: 99999
}
{
  _id: ObjectId('6a187f82caf8cae267f42511'),
  value: 99998
}
{
  _id: ObjectId('6a187f82caf8cae267f42510'),
  value: 99997
}
{
  _id: ObjectId('6a187f82caf8cae267f4250f'),
  value: 99996
}
```

Анализ плана запроса с индексом

```
executionTimeMillis: 0,
```

→ Сравнение времени выполнения: После создания индекса для поля **value** время выполнения запроса уменьшилось. Запрос с индексом оказался более эффективным, так как MongoDB использовала индекс для поиска и сортировки документов вместо полного сканирования коллекции.

4. Выводы

В ходе лабораторной работы были изучены основные возможности системы управления базами данных MongoDB и принципы работы с документно - ориентированными базами данных.

В процессе выполнения работы были освоены методы создания и удаления баз данных и коллекций, а также добавление документов с помощью методов `insertOne()` и `insertMany()`.

Были выполнены различные операции выборки данных с использованием методов `find()`, `findOne()`, `sort()`, `limit()` и проекций полей. Также были изучены логические операторы `$gte`, `$lte`, `$exists`, `$all` и оператор `$slice` для работы с массивами.

Были исследованы возможности агрегированных запросов, курсоров и JavaScript-функций в MongoDB. Для обработки результатов запросов использовались методы `forEach()` и `aggregate()`.

Кроме того, были освоены операции обновления документов с помощью операторов `$set`, `$push`, `$inc` и `$unset`, а также удаление документов методами `deleteOne()` и `deleteMany()`.

Дополнительно были изучены связанные документы с использованием `DBRef`, создание индексов для ускорения поиска данных и анализ выполнения запросов при помощи метода `explain()`.

В результате выполнения лабораторной работы были получены практические навыки работы с MongoDB и изучены основные методы управления документами и коллекциями в NoSQL базах данных.